

FOC Firmware Overview

Power Application Controller®





TABLE OF CONTENTS

Table of Contents	2
1 Introduction	3
2 Firmware Overview	3
2.1 Features	3
2.2 Motor Position and Speed	3
2.3 Behavioral Description	4
2.3.1 Open Loop Mode.....	4
2.3.2 Closed Loop Control	4
2.3.3 Motor Stopping Modes	6
2.4 PWM Generation and ADC Sampling	7
3 Firmware Architecture.....	9
3.1 Initialization.....	10
3.2 Interrupt Service Routines.....	10
3.2.1 Timer A ISR.....	10
3.2.2 ADC ISR.....	11
3.2.3 SysTick ISR	12
3.2.4 UART ISR.....	13
3.2.5 GPIO ISR	13
3.3 State Machines.....	14
3.3.1 Main State Machine	14
3.3.2 Sub-State Machines of the Main State Machine	17
3.3.3 AppControl/Control State Machines.....	17
3.4 FOC Core Functions	20
Contact Information.....	21
Important Notice	21



1 INTRODUCTION

This document provides details of the PAC FOC Firmware, which implements software based Field Oriented Control (FOC) for the PMSM class of motors. The document gives an overview of the features and FOC modes implemented by the firmware, followed by details of the software architecture.

2 FIRMWARE OVERVIEW

2.1 Features

One of the key advantages of the PAC architecture is the hardware integration, which enables the peripherals to support tasks without firmware intervention; an example is the ADC sequencer which orchestrates the high-precision ADC sampling. This allows a simpler hardware design and lets the firmware focus on high-value features, such as the control loop for FOC.

The FOC firmware supports the following features:

- Supports electrical frequency up to 3000 Hz
- Supports a Control loop up to 30 kHz
- Uses space-vector modulation (SVPWM) to drive a PMSM motor
- Configurable PWM frequency
- Can be operated in speed, torque, or power control modes
- Various sensor and sensor-less motor position and speed calculators available
- Field weakening to achieve higher speeds
- Open Phase Detection (OPD) prevents motor from starting when wiring of any phase is broken
- Various fault detection mechanisms to prevent motor from running in problematic conditions
- Debug mode allows viewing internal algorithm variables in real-time on an oscilloscope.
 - Especially useful for tuning control algorithm parameters.
- UART based Graphic User Interface (GUI) for user control and debug of motor operation
- Automatic motor parameter measurement and PI controller tuning

2.2 Motor Position and Speed

The FOC algorithms require the motor position and speed for calculation. In some applications, this information can be obtained by using position sensors. However, this method increases the cost and compromises reliability because these sensors can fail. The use of sensors may not even be possible in some applications like compressors. Nevertheless, the applications that do use sensors will find that the PAC platform supports this method with the availability of GPIO's that receive the sensor signals.

This firmware package supports a number of motor position (MPOS) options, both sensor and sensor-less. The user is able to select between the following MPOS options:

- 1) Sensorless Estimator with Align&Go startup
- 2) Sensorless Estimator with Hall Sensor startup
- 3) Hall Sensor Only
- 4) QEP Encoder with Align&Go startup

For more information on these startup methods and how to configure them, please refer to the MPOS Guides that are available.

2.3 Behavioral Description

2.3.1 Open Loop Mode

All the MPOS methods that use Align&Go as a startup mechanism require the use of the open-loop mode. When in open-loop mode, the firmware generates its own forced angle to drive the motor. The block diagram in Figure 1 illustrates how the FOC control loop is implemented during the startup state.

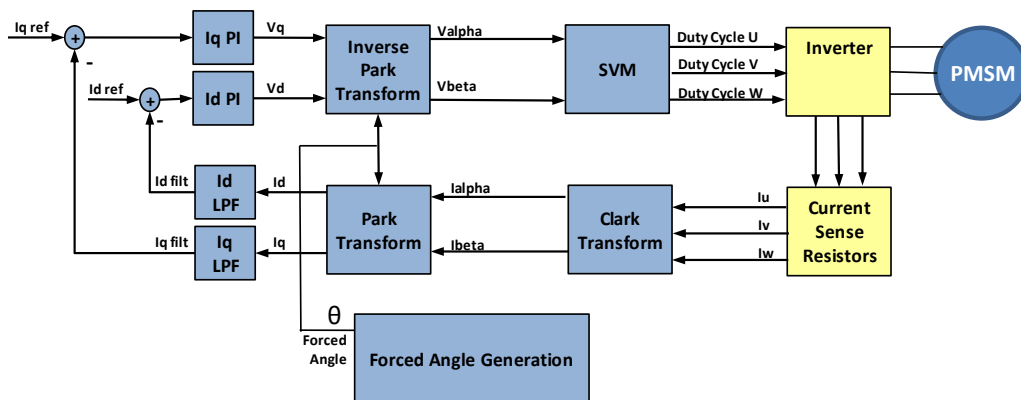


Figure 1: Open Loop Mode Block Diagram

2.3.2 Closed Loop Control

Once the motor is in closed loop, operation it relies on the rotor position information provided by the MPOS module that was selected by the user. Depending on the application, the motor can be driven in either speed control, torque control, or power control modes. The speed information from the MPOS module is not used during closed loop torque control or power control modes.

2.3.2.1 Speed Control Mode

In speed control mode a speed PI controller is used to regulate the torque required to maintain the desired speed target. The controller regulates the speed dynamically in response to changes in the motor by using the speed information from the MPOS module as shown in Figure 2. If the motor load changes, the speed will change as a result, and the speed controller will adjust the target torque current accordingly.

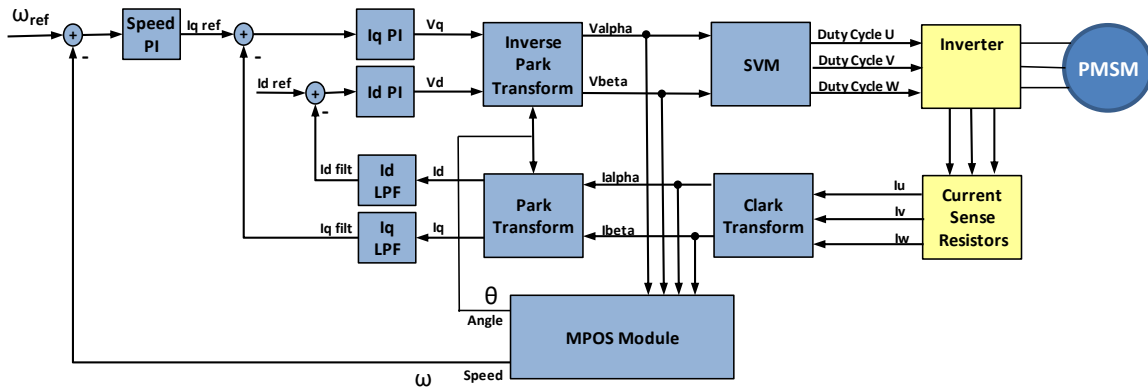


Figure 2: Speed Control Block Diagram

Speed control mode is commonly used in applications such as fans and compressors.

2.3.2.2 Torque Control Mode

In torque control mode, there is no speed controller regulating the torque current. Instead, the target torque current is set externally and the system will regulate the torque that the motor applies to the load. The final speed of the motor will depend on the speed at which the applied torque reaches equilibrium with the force the load applies back to the motor. It will not respond to load changes, so the speed will reduce if the load increases.

Examples of cases in which torque control may be desired are electric bikes and some fans or blowers.

The block diagram Figure 3 illustrates the control loop in torque control mode.

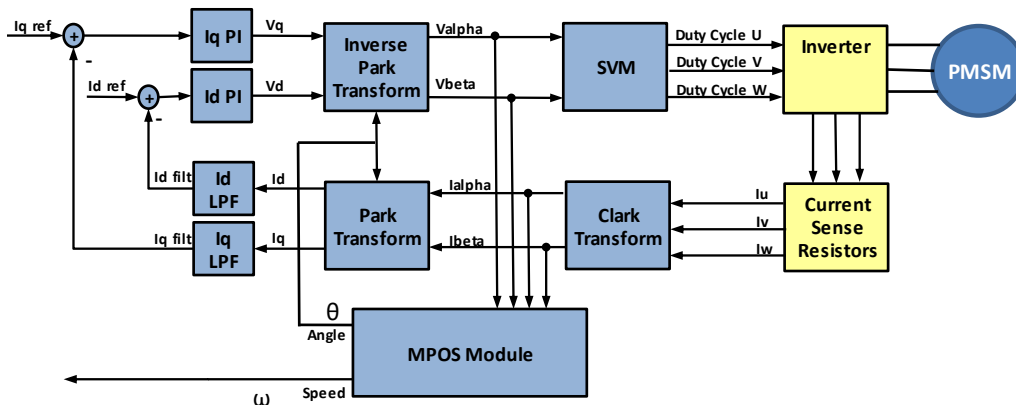


Figure 3: Torque Control Block Diagram

2.3.2.3 Power Control Mode

In power control mode a speed PI controller is used to regulate the power of the motor as shown in

Figure 4. An instantaneous power measurement is calculated from the phase currents and voltages and used as the feedback signal for the control loop.

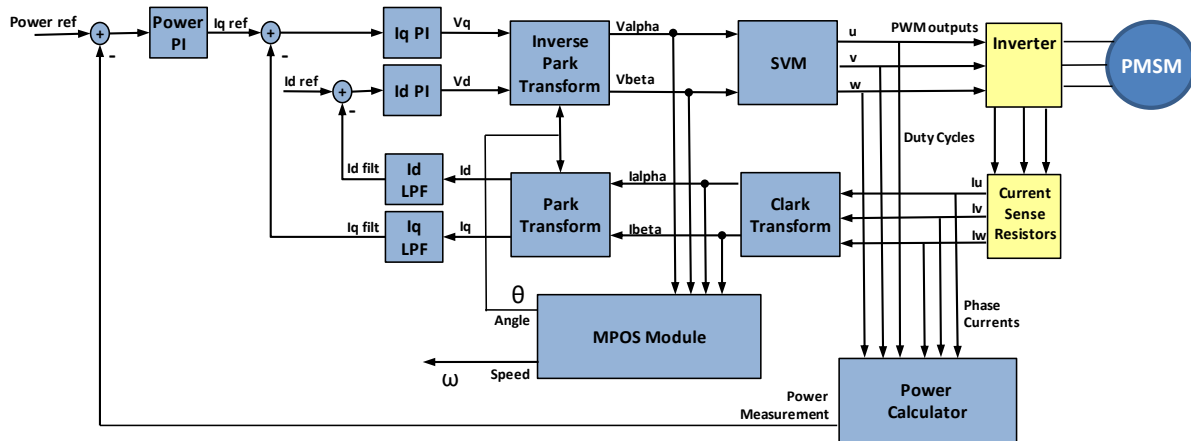


Figure 4: Power Control Block Diagram

Power control mode is commonly used in applications such as vacuum cleaners.

2.3.3 Motor Stopping Modes

There are various ways of disabling the motor depending on the circumstances:

1) Disabled:

The Disabled state is entered when:

- The motor disable command is issued by the user while the motor is operating in open-loop mode
- A fault occurs.

When a command to disable the motor is issued (`motor_enable_cmd = 0`), the driver outputs are immediately disabled.

2) Coasting Stop:

When a motor disable command is issued by the user while the motor is operating in closed-loop mode, the motor controller enters the Coasting mode which is followed by the Driver-Disable stop mode as soon as the speed is detected to be under the minimum speed.

During the coasting state, FOC is still active but operating in torque mode with $I_q=0$. While the motor is slowing down, if a motor enable command is issued and the speed is above the OL-to-CL switchover speed, then one of the following actions will be taken depending on the value of the `speed_mode_flag`:

- `speed_mode_flag=1`:
Operation is set to CL speed mode and the speed ramp value is set to the estimated speed.
- `speed_mode_flag=0`:
Operation is set to CL torque mode and I_q_ref is restored to the I_q_ref value when the stop command was issued.

If while coasting, the motor slows down to reach the OL-to-CL switchover speed before receiving an enable command from the user, then the state machine will enter the Disabled state and all driver outputs will be disabled.

If an enable command is issued after the Disabled state is entered, the startup is then handled through the standard startup sequence.

2.4 PWM Generation and ADC Sampling

The figure below illustrates how Timer A is used to generate the PWM signals that drive the motor. It also shows that CTR0 of Timer A is used to trigger the ADC sequencer, which samples the various signals needed for FOC.

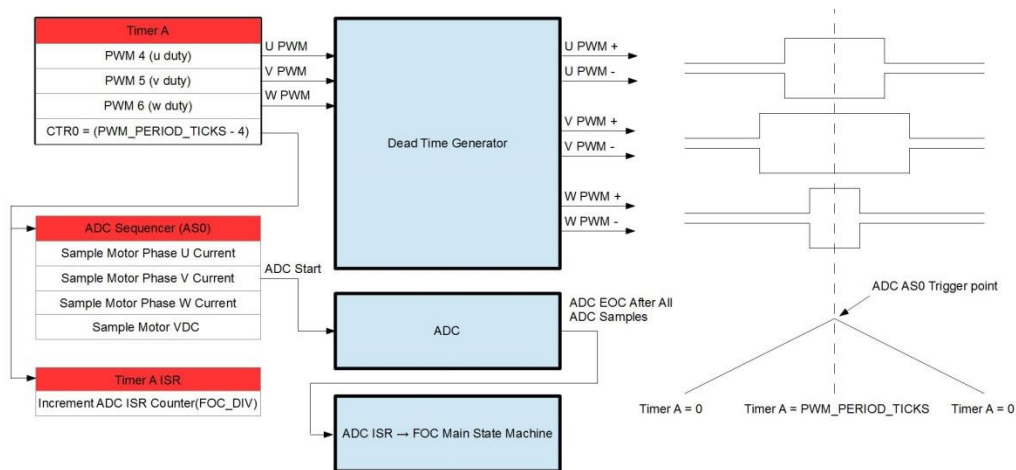


Figure 5: PWM ADC Sequencer Interaction

For the following bulleted items, refer to both the figure above and the more detailed figure below.

- Timer A is in charge of generating the PWM signals for driving the 3-phase H-Bridge.
- The ADC Sequencer triggers at $\text{TimerA} = \text{CTR0} = (\text{PWM_PERIOD_TICKS} - 4)$.
 - $\text{PWM_PERIOD_TICKS} - 4$ is used because issues were seen previously when using a number close to PWM_PERIOD_TICKS
- The ADC digitizes 3 (or 2) phase currents and VDC.
- After the 3 phase currents are converted, the ADC Sequencer triggers the ADC ISR, which invokes the Control State Machine. The ADC sequencer continues to convert the remaining ADC samples specified by the sequence. This allows the control state machine to begin processing the currents as soon as possible.
- The Control State Machine runs at the frequency determined by the `foc_ctrl_divider` by using the TimerA ISR counter, which gets incremented by the Timer A ISR.

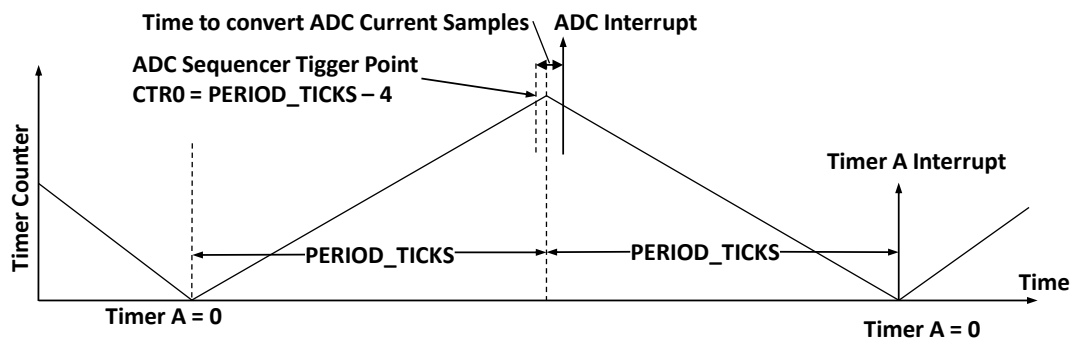
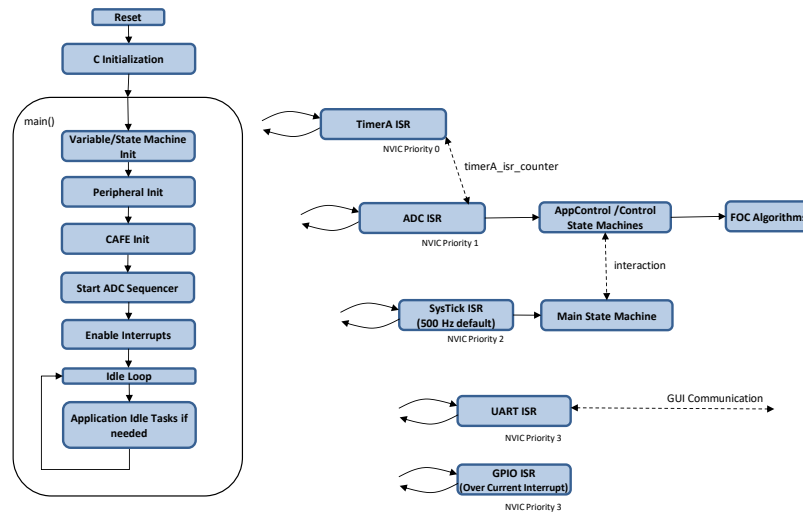


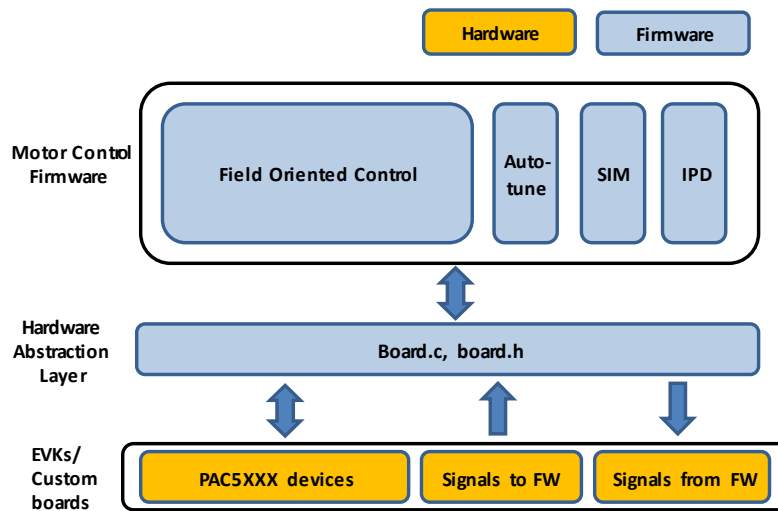
Figure 6: Detailed PWM ADC Sequencer Interaction

3 FIRMWARE ARCHITECTURE

The top level architecture of the FOC Firmware is shown in the figure below. Initialization is performed in main() after which interrupts are enabled to allow the various interrupt service routines (ISRs) to begin. The main() ends with an infinite while loop (idle loop) because the rest of the application is executed in the ISRs. Optional application specific tasks can be added to this idle loop.



(a) Flow chart



(b) Block diagram

Figure 7: Top Level FOC Firmware Architecture

3.1 Initialization

After the MCU comes out of reset, it will begin the initialization process:

- 1) The reset handler calls the C environment initialization first before calling main().
- 2) In main(), initialization of the system is performed for the following:
 - a. Variable Initialization
 - b. State Machine Initialization
 - c. Peripheral Initialization
 - d. Configurable AFE (CAFE) Initialization
- 3) The ADC Sequencer is started
- 4) Interrupts are enabled which starts running the state machines that are called from the ISRs.

Once the initialization process is complete, the firmware is ready to accept commands from the user. These commands may be sent through the GUI (via UART commands) or through other communication protocols that the user may implement in the application layer.

3.2 Interrupt Service Routines

The Interrupt Service Routines (ISRs) that are used in this application are:

- Timer A ISR- Priority 0
 - PWM period interrupt
- ADC ISR - Priority 1
 - Executes the Control State Machine
- SysTick ISR - Priority 2
 - Executes the Main State Machine
- UART ISR - Priority 3
 - Communicates with the GUI
- GPIO ISR - Priority 3
 - Handles the Over Current (OC) event

3.2.1 Timer A ISR

The Timer A ISR occurs every PWM up/down cycle and is used to count the number of PWM cycles. This counter is then used by the ADC ISR to control the frequency ratio between the control frequency and the PWM cycle frequency by running the control loop every specified number of PWM cycles.

The Timer A ISR simply increments a counter, then clears the interrupt flag and exits.

TimerA ISR is the highest priority interrupt with Priority=0. This means that even if the ADC interrupt, or any other interrupt, is being serviced, it will pre-empt the MCU so that the counter will be incremented. This is critical for proper operation of the control loop and application.

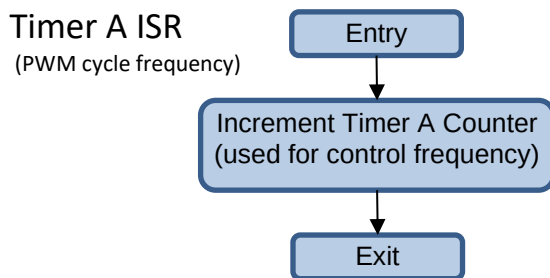


Figure 8: Timer A ISR

3.2.2 ADC ISR

The ADC ISR occurs when an ADC conversion sequence is complete. The primary function of the ISR is to execute the Control state machine which calls algorithms that operate on the ADC readings and outputs new duty cycles to the motor drivers. The FOC algorithm processing is called during certain Control state machine states.

The ADC uses its auto-sequences for converting several critical parameters as the motor is running (current, bus voltage, etc.). This happens at the PWM frequency, so that these samples are always taken in the same place in the PWM waveform.

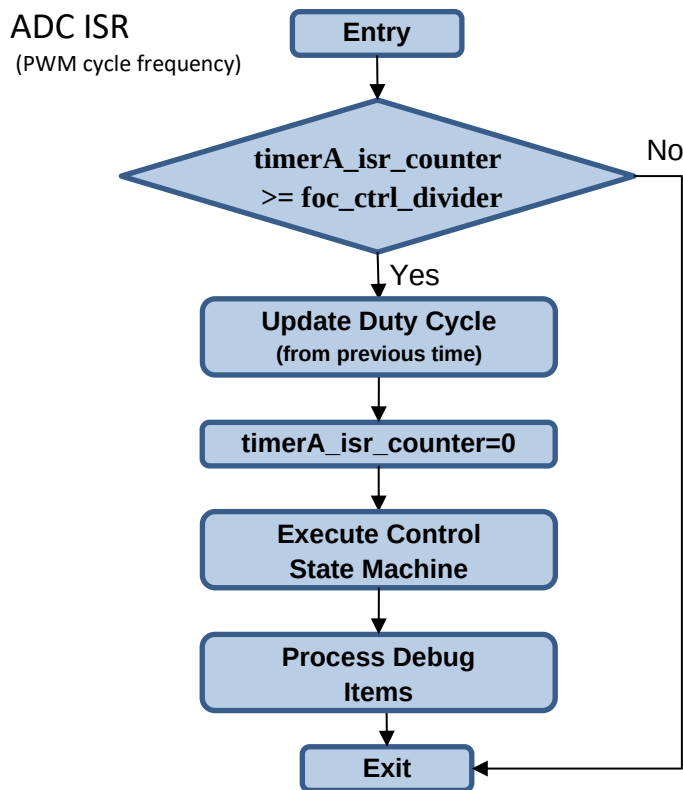


Figure 9: ADC ISR

The ADC ISR first checks the Timer A counter to see if enough PWM periods have elapsed before executing the rest of the functionality. Then it performs the following:

- Update the duty cycles for the 3 phase PWMs
- Reset the TimerA counter to start over
- Execute the Control state machine
- Update any Debug items that are enabled

The ADC interrupt priority is set to Priority=1. This allows the control loop to run uninterrupted, except by the Timer A interrupt.

3.2.3 SysTick ISR

The SysTick ISR is responsible for executing the Main state machine, processing for motor position (mpos), and performing checks/fault detection/reporting. The Main state machine controls the application layer, which orchestrates motor operation through interaction with the Control state machine.

The SysTick ISR is set to Priority=2. The functions executed by the SysTick ISR will run only when the ADC ISR (Control loop) and Timer A ISR are not running.

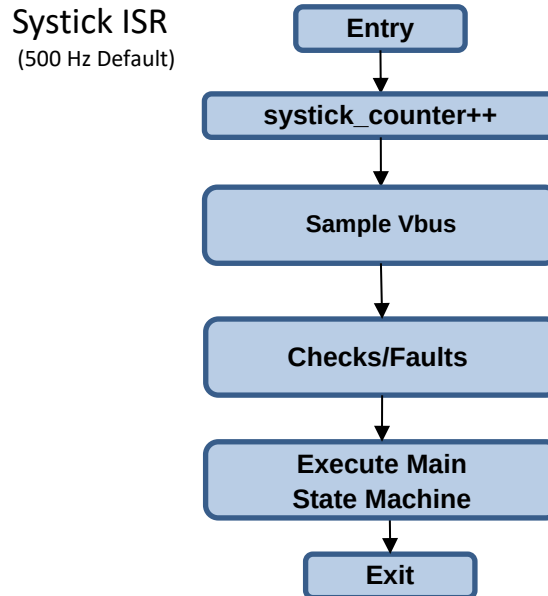


Figure 10: Systick ISR

3.2.4 UART ISR

The UART ISR handles communication to an external host, in this case, the Windows GUI. The GUI communicates with the target PAC using a USB to UART adapter.

When processing receive characters, the firmware assembles them into a single message, then acts on the message (reads or writes data to the target). When the message has been completely received, the firmware turns off the receive interrupt, and turns on the transmit interrupt so that the response may be sent to the host. When all of the transmit characters have been sent out, then the transmit interrupt is disabled and the receive interrupt is turned back on.

The UART ISR is set to Priority=3. These functions are not time critical, and the priority allows them to run when there is time available so that it won't affect the operation of the other critical FOC functions.

3.2.5 GPIO ISR

The firmware application configures the analog sub-system to provide over-current protection. When an over-current event occurs, an interrupt is generated on a GPIO. The application uses the GPIO ISR to handle such events.

If an interrupt is received on the GPIO, the firmware sets the FAULT_CAFE_OC fault. The Main state machine can then take action based on this flag.

When the over-current event occurs, the gate drivers are latched off automatically by the hardware without software intervention. The user may reset this via a UART command; otherwise, the device will remain non-operational until a power cycle.

The GPIO ISR is set to Priority=3.

3.3 State Machines

3.3.1 Main State Machine

The Main State Machine, `statemachine_appmain()`, is the top-level state machine that handles transitions between different motor states, such as stopped, open-loop startup, and closed-loop. It also takes care of disabling the motor as part of the fault handling. In addition, the Main state machine is responsible for initialization of motor parameters and any Power-On Self-Test (POST) procedures that need to be carried out on system power-up. If additional application layer aspects need to be added to the system, they would most likely be handled by the Main state machine.

Figure 11 represents the Main state machine. The Main state machine is called by the SysTick ISR and runs at the SysTick frequency. The state changes do not need to be updated at the control frequency so running them at a lower frequency makes more processor time available for the more critical control loop. Additionally, because the functions carried out by this state machine are not as time-critical as the control loop, the Main state machine and its sub-state machines execute from Flash memory instead of RAM.

A description of each of the states follows:

RESET State

The first time the Main state machine is executed, it starts in the RESET state. Under this state the default motor parameters are initialized, system variables are reset along with sub-state machine states.

POST State

The *POST* state immediately follows the *RESET* state and handles Power On Self-Test (POST) functions. It executes the ADC offset calibration sequence and can perform additional tests added by the user. When complete, the state machine advances to the *DISABLE* state.

DISABLE State

The *DISABLE* state is entered every time the motor is disabled. It runs through the sequence that disables the outputs. This state is a transitional state which runs once when the motor is disabled and transitions immediately to the *STOPPED* state.

MAIN

State Machine

Called by: SysTick ISR

Frequency: SysTick

Memory: Flash

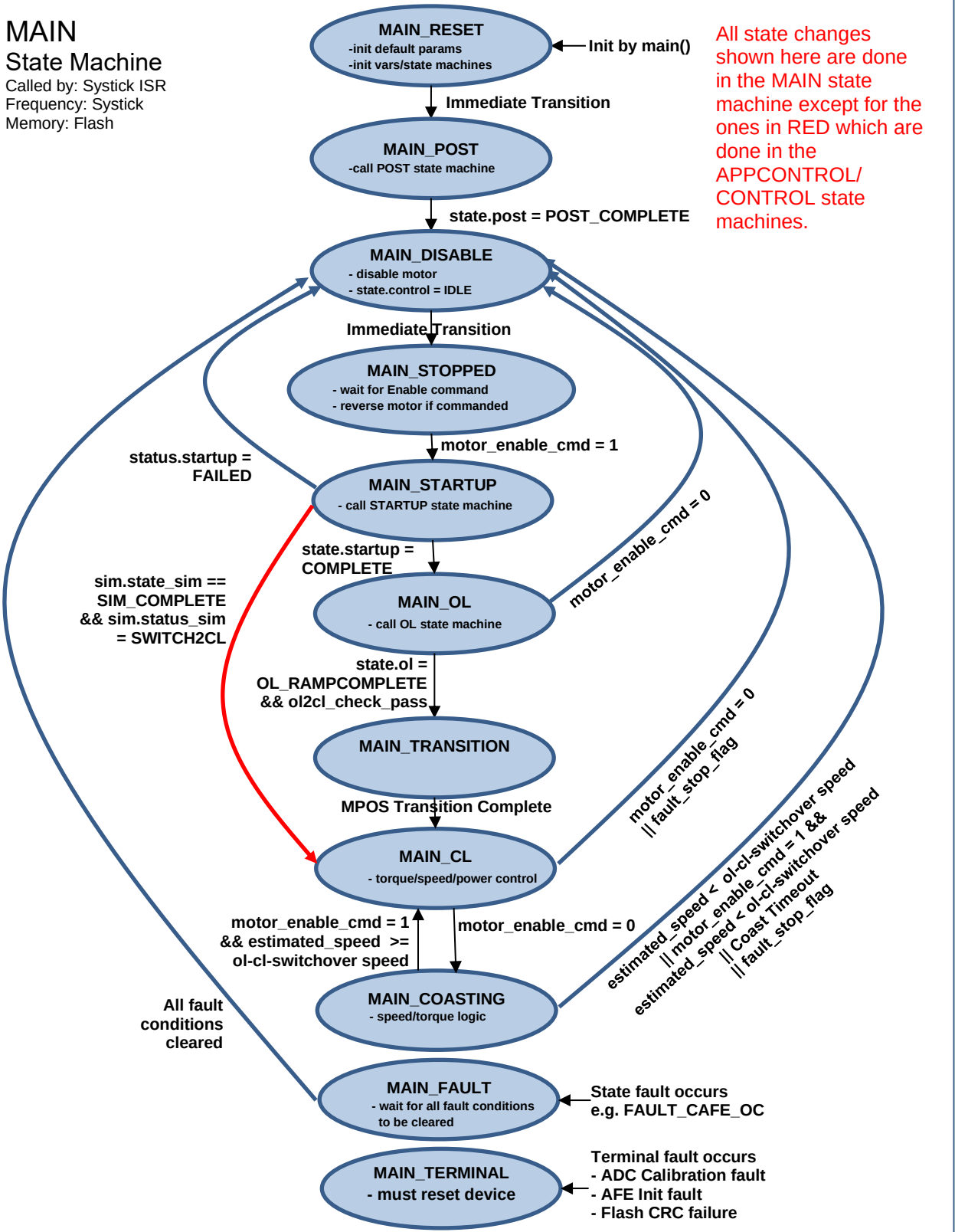


Figure 11 MAIN State Machine

STOPPED State

In this state, the system waits until commanded to enable the motor, usually via a UART message. When the enable command is received, the state machine advances to the *STARTUP* state. In the stopped state, a command to reverse directions is also processed.

STARTUP State

The *STARTUP* state runs through the sequence necessary to start running the motor. The startup sequence is controlled by the Startup state machine. If the application has features like Start-In-Motion (SIM) or Initial Position Detection (IPD) enabled, they are executed from the Startup state machine. When the Startup state machine is complete, the Main state machine advances to the *OL* state.

OL (Open-Loop) State

Under the *OL* state, the motor follows the Open Loop sequence controlled by the OL state machine. This sequence in its most basic form consists of forcing the motor angle through an align stage, followed by a speed ramp to the desired open loop speed. Once the ramp is complete the state machine is ready to advance into the *TRANSITION* state, but it will only advance to that state if Auto Closed Loop is enabled and all the open loop to closed loop checks are satisfied.

TRANSITION State

In this state, FOC is prepared for the transition to Closed Loop state and at the proper time, the state machine advances to the Closed-Loop State

CL (Closed-Loop) State

In the *CL* state, the system is controlling the motor in closed-loop mode using the estimated angle and speed. This is the state the motor normally operates in until the motor is disabled.

COASTING State

When the motor is disabled while operating in the *CL* state, it enters the *COASTING* state. This state is a transition state before reaching the *DISABLE* state. In the *COASTING* state, the motor is still operating in closed-loop mode but slowing down by coasting. If the motor is re-enabled before the CL minimum speed is reached, then the state machine will transition directly back to the *CL* state. Otherwise, if the speed drops below the CL minimum, then the Main state machine proceeds to the *DISABLE* state where it will initiate the sequence to disable the motor and prepare it for the next time it gets enabled.

FAULT State

In this state, the system waits until the `fault_oc_flag` is cleared by a GUI command. Once cleared, the Main state machine transitions to the *DISABLE* state.

Several operational conditions may cause the application to exit some of these states, such as faults. In this case the natural flow of the main state machine described above will be interrupted and re-directed to enter the *DISABLE* state in order to stop running the motor.

TERMINAL State

This state is entered if a terminal fault is encountered that the application can't recover from, Examples are ADC Calibration Fault, AFE Init Fault, Flash CRC Failure.

3.3.2 Sub-State Machines of the Main State Machine

The following diagrams show the sub-state machines that are called from the Main state machine. The names of the sub-state machines correspond to the MAIN state they are called from.

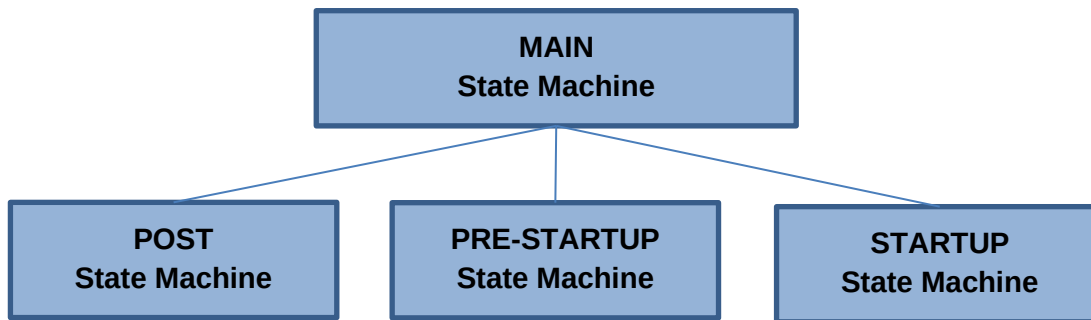


Figure 12: Main Sub State Machines

3.3.3 AppControl/Control State Machines

The APPCONTROL & CONTROL State Machines manage what functions need to be executed during the control loop. From here on, we will refer to them as the Control state machines. The control loop contains the different sequences to follow during the modes of motor operation: open-loop, closed-loop speed mode, and closed-loop torque mode. It also defines the sequences that get called during other modes of operation such as during ADC offset calibration or while certain features like Start-In-Motion (SIM) are running.

Figure 13 represents the Control state machines. These State Machines are called inside the ADC ISR and run at the predetermined control frequency. The Control state machines are resident in RAM because the ability to execute the control loop as quickly as possible maximizes the control frequency.

It's very important to note that most of the state changes that occur in these state machines are controlled by the Main state machine and Main sub-state machines. The only exceptions are some state changes that cannot be handled by the Main State Machine because they must happen without a delay, and thus, are handled instead by the Control state machines. Likewise, there are some state changes of the Main State Machine that are handled in the Control state machines because they are time-critical and cannot be handled at the slower rate. Although, it may be unusual for the state changes of the Control state machines and the Main state machine to be intertwined in this way, this technique provides us with a large gain in available processor execution time. As mentioned earlier, the structure of the FOC firmware is centered around keeping only the time critical functions in the control loop and moving all



other functions into the Main state machine tree. Using this technique not only helps to maximize the control loop frequency, but also helps to reduce the total RAM used.

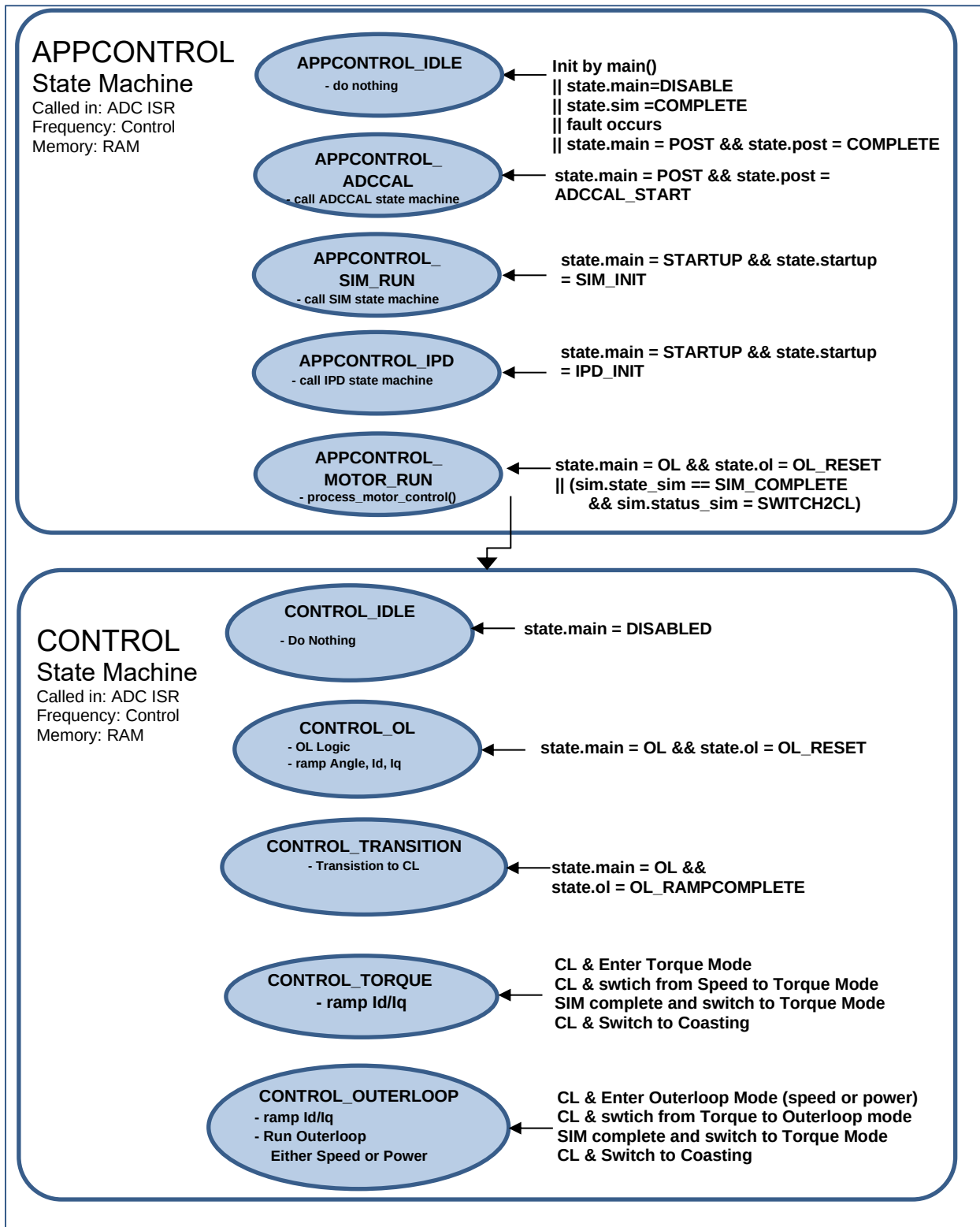


Figure 13: Control State Machines

3.4 FOC Core Functions

Depending on the control sequence path that is selected, the Control State Machines may call the `process_foc()` function, which runs the core functional blocks of the FOC algorithm:

- Clark transform
- Park Transform
- Iq/Id Filtering
- Iq/Id PI controllers
- Inverse Park
- SVM

The figure below illustrates a flowchart of the `process_foc()` function.

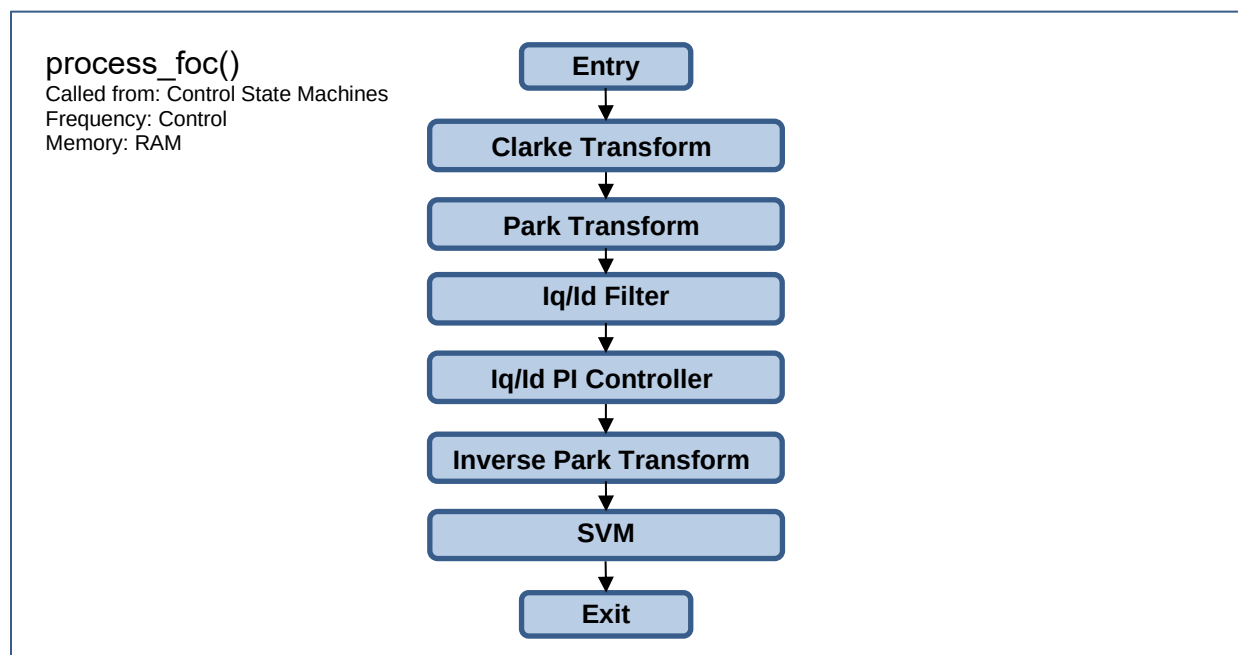


Figure 14: FOC Algorithm Flow



CONTACT INFORMATION

For the latest specifications, additional product information, worldwide sales and distribution locations:

Web: www.qorvo.com

Tel: 1-844-890-8163

Email: customer.support@qorvo.com

IMPORTANT NOTICE

The information contained herein is believed to be reliable; however, Qorvo makes no warranties regarding the information contained herein and assumes no responsibility or liability whatsoever for the use of the information contained herein. All information contained herein is subject to change without notice. Customers should obtain and verify the latest relevant information before placing orders for Qorvo products. The information contained herein or any use of such information does not grant, explicitly or implicitly, to any party any patent rights, licenses, or any other intellectual property rights, whether with regard to such information itself or anything described by such information. **THIS INFORMATION DOES NOT CONSTITUTE A WARRANTY WITH RESPECT TO THE PRODUCTS DESCRIBED HEREIN, AND QORVO HEREBY DISCLAIMS ANY AND ALL WARRANTIES WITH RESPECT TO SUCH PRODUCTS WHETHER EXPRESS OR IMPLIED BY LAW, COURSE OF DEALING, COURSE OF PERFORMANCE, USAGE OF TRADE OR OTHERWISE, INCLUDING THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.**

Without limiting the generality of the foregoing, Qorvo products are not warranted or authorized for use as critical components in medical, life-saving, or life-sustaining applications, or other applications where a failure would reasonably be expected to cause severe personal injury or death.

Copyright 2019 © Qorvo, Inc. | Qorvo is a registered trademark of Qorvo, Inc.